🧠 DSA Notes

Analysis of Recursion — Beginner's Cheatsheet

📌 1. Why Recursive Analysis is Different from Iterative

With loops, you count iterations. With recursion, you cannot — because each call spawns more calls. You need a different tool: the Recurrence Relation.

Iterative Analysis	Recursive Analysis
Count loop iterations directly	Write a Recurrence Relation first
e.g. for loop runs n times → O(n)	e.g. T(n) = 2T(n/2) + O(1)
No recursive calls to track	Each call adds sub-problems to track
Straightforward to analyse	Solve using substitution / recursion tree / Master theorem

📌 2. What is a Recurrence Relation?

💡 What is a Recurrence Relation?

A recurrence relation is an equation that describes T(n) — the time taken by a recursive function — in terms of a smaller input.

It has two parts:

- **Recursive case:** What happens when n > base case? (the repeating formula)
- **Base case:** When does recursion stop? (usually T(0) or T(1) = O(1))

Analogy: Think of it like a recipe for a recipe. 'To make a dish of size n, I first make two dishes of size n/2, plus a little extra work.'

General form:

```
T(n) = [number of sub-calls] × T(size of each sub-call) + [work
done outside calls]
T(base case) = O(1)
```

Real example from Image 1 — fun(n/2) called twice, print done once:

```
T(n) = T(n/2) + T(n/2) + O(1)
T(n) = 2T(n/2) + O(1)          ← simplified
T(0) = O(1)                     ← base case
```

📌 3. How to Write a Recurrence Relation — 3 Steps

✅ 3-Step Method to Write a Recurrence Relation

1. **Define T(n) —** Let T(n) = time taken by the function for input of size n.
2. **Write the recursive case —** Count sub-calls and their input sizes. Add work done in the current call (loops, prints = O(n) or O(1)).
3. **Write the base case —** Find when recursion stops (if n <= 0, if n <= 1 etc.) — that is T(0) = O(1) or T(1) = O(1).

📌 4. Example 1 — Two Recursive Calls (n/2 each)

The Code:

```
void fun(int n) {
  if (n <= 0) return;          // base case
  print('GfG');                // O(1) work
  fun(n/2);                    // recursive call 1
  fun(n/2);                    // recursive call 2
}
```

Step 1 — Identify what each line costs and write the Recurrence Relation:

```
T(n) = T(n/2) + T(n/2) + O(1)
```

```
T(n) = 2T(n/2) + O(1)          ← two calls of half size + constant work
T(0) = O(1)                    ← base case: just a return
```

Step 2 — Understand it in plain English:

> **This function calls itself TWICE each time, but each call works on HALF the input (n/2). The O(1) is the print statement — it takes constant time. Combined: 2T(n/2) + O(1). Solving this gives O(n).**

Final Answer	Complexity = O(n)

📌 5. Example 2 — Loop + Two Recursive Calls (n/2, n/3)

The Code:

```
void fun(int n) {
  if (n <= 0) return;                    // base case
  for (int i = 0; i < n; i++)            // O(n) work
    print('GfG');
  fun(n/2);                              // recursive call 1
  fun(n/3);                              // recursive call 2
}
```

Step 1 — Identify what each line costs and write the Recurrence Relation:

```
T(n) = T(n/2) + T(n/3) + O(n)
                                  ← two calls of DIFFERENT sizes + a loop
T(0) = O(1)                       ← base case
```

Step 2 — Understand it in plain English:

> **Now there is a for-loop that runs n times — that costs O(n) per call (not O(1) like before). Plus two recursive calls with different sizes: n/2 and n/3. The recurrence is T(n) = T(n/2) + T(n/3) + O(n). This is harder to solve manually — typically done with a recursion tree.**

Final Answer	Complexity = O(n log n) approximately

📌 6. Example 3 — Single Recursive Call (n-1)

The Code:

```
void fun(int n) {
  if (n <= 1) return;          // base case
  print('GfG');                // O(1) work
  fun(n - 1);                  // recursive call — reduces by 1
}
```

Step 1 — Identify what each line costs and write the Recurrence Relation:

```
T(n) = T(n-1) + O(1)
                          ← one call on (n-1) + constant work
T(1) = O(1)               ← base case
```

Step 2 — Understand it in plain English:

> **Each call does O(1) work (just a print) and calls itself with n-1. So the function runs n times in a chain: fun(n) → fun(n-1) → fun(n-2) → ... → fun(1). Total work = n × O(1) = O(n). This is the simplest recurrence — a linear chain.**

Final Answer	Complexity = O(n)

📌 7. How to Read Any Recurrence Relation

Example: T(n) = 2T(n/2) + O(1)

```
T(n)  =  2T(n/2)  +  O(1)
```

T(n)	Time for input of size n — the thing we are trying to find

`2T(n/2)`	Two recursive calls, each with input of size n/2
`+ O(1)`	Extra work done in this call (here: one print statement = constant time)
`T(0) = O(1)`	Base case — when n <= 0, just return. That takes constant time.

Example: T(n) = T(n/2) + T(n/3) + O(n)

```
T(n)  =  T(n/2)  +  T(n/3)  +  O(n)
```

`T(n/2)`	First recursive call on half the input
`T(n/3)`	Second recursive call on one-third of the input
`O(n)`	A for-loop that runs n times in this call — linear extra work

> 📌 **Key Insight: The terms INSIDE T(...) tell you the sub-problem size. The term OUTSIDE T(...) tells you the work done at the current level.**

📌 8. Quick Reference — Common Recurrences

Recurrence	Pattern	Complexity
`T(n) = T(n-1) + O(1)`	Linear chain (Example 3)	`O(n)`
`T(n) = 2T(n/2) + O(1)`	Two halves + const (Ex. 1)	`O(n)`
`T(n) = 2T(n/2) + O(n)`	Merge sort pattern	`O(n log n)`
`T(n) = T(n/2) + O(1)`	Binary search pattern	`O(log n)`
`T(n) = T(n/2)+T(n/3)+O(n)`	Two diff sizes (Example 2)	`O(n log n) ~`

🎯 9. Final Takeaway

🎯 Final Takeaway

- **Recursive functions** cannot be analysed by counting loop steps alone.
- **Write a Recurrence Relation** first — it captures what the function does at each level.
- **Base case = T(0) or T(1) = O(1)** — it defines where recursion stops.
- **Solve the recurrence** using substitution, recursion tree, or Master theorem.
- **The final T(n) expression** gives you the Big-O complexity of the recursive function.